Tutorial4

Teaching Team of STA314

Roadmap and learning goals

In this lab, we demonstrate nonlinear models using the `Wage` data. We investigate how wages vary with age, using **R** throughout. By the end of the lab, you should be able to:

- Fit and interpret nonlinear regression curves.
- Explain the difference between polynomial regression, regression splines, and natural splines.
- Fit generalized additive models (GAMs) with smooth terms and categorical predictors.
- Explain how the smoothing parameter controls flexibility.
- Use LOWESS to fit a local regression curve.
- Compare model complexity using degrees of freedom, smoothing parameters, and validation or cross-validation.

Packages and data

We will use `ISLR2` for the data, `splines` for spline bases, and `mgcv` for generalized additive models. The `stats` package (included with R) supplies `lm()`, `predict()`, and `loess()`.

```
# Run this once if the packages are not installed:
# install.packages(c("ISLR2", "mgcv"))

library(ISLR2)
library(splines)
library(mgcv)

knitr::opts_chunk$set(echo = TRUE, fig.width = 8, fig.height = 6)
```

```
data("Wage", package = "ISLR2")
y <- Wage$wage
age <- Wage$age
head(Wage, 3)
```

```
##        year age           maritl     race       education             region
## 231655 2006  18 1. Never Married 1. White    1. < HS Grad 2. Middle Atlantic
## 86582  2004  24 1. Never Married 1. White 4. College Grad 2. Middle Atlantic
## 161300 2003  45       2. Married 1. White 3. Some College 2. Middle Atlantic
##              jobclass          health health_ins  logwage      wage
## 231655  1. Industrial      1. <=Good      2. No 4.318063  75.04315
## 86582  2. Information 2. >=Very Good      2. No 4.255273  70.47602
## 161300  1. Industrial      1. <=Good     1. Yes 4.875061 130.98218
```

```
age_grid <- seq(min(age), max(age), length.out = 100)
age_df <- data.frame(age = age_grid)
```

We will model `wage` as a function of `age` for most of the lab. We also use `year` and `education` when we fit additive models with several predictors.

Polynomial regression

A degree-d polynomial regression is a linear regression in the transformed predictors:

$$Y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \varepsilon_i.$$

Conceptually: `age` $\longrightarrow [1, \text{age}, \text{age}^2, \ldots, \text{age}^d] \longrightarrow$ linear regression $\longrightarrow$ predicted wage.

Higher polynomial degrees increase flexibility, but can behave poorly near the boundaries. We begin with a fourth-degree polynomial, using `raw = TRUE` to work with ordinary powers of age.

```
poly_fit <- lm(wage ~ poly(age, degree = 4, raw = TRUE), data = Wage)
summary(poly_fit)
```

```
##
## Call:
## lm(formula = wage ~ poly(age, degree = 4, raw = TRUE), data = Wage)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -98.707 -24.626  -4.993  15.217 203.693
##
## Coefficients:
##                                        Estimate Std. Error t value Pr(>|t|)
## (Intercept)                          -1.842e+02  6.004e+01  -3.067 0.002180 **
## poly(age, degree = 4, raw = TRUE)1  2.125e+01  5.887e+00   3.609 0.000312 ***
## poly(age, degree = 4, raw = TRUE)2 -5.639e-01  2.061e-01  -2.736 0.006261 **
## poly(age, degree = 4, raw = TRUE)3  6.811e-03  3.066e-03   2.221 0.026398 *
## poly(age, degree = 4, raw = TRUE)4 -3.204e-05  1.641e-05  -1.952 0.051039 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 39.91 on 2995 degrees of freedom
## Multiple R-squared:  0.08626,    Adjusted R-squared:  0.08504
## F-statistic: 70.69 on 4 and 2995 DF,  p-value: < 2.2e-16
```

Plotting fitted curves and confidence bands

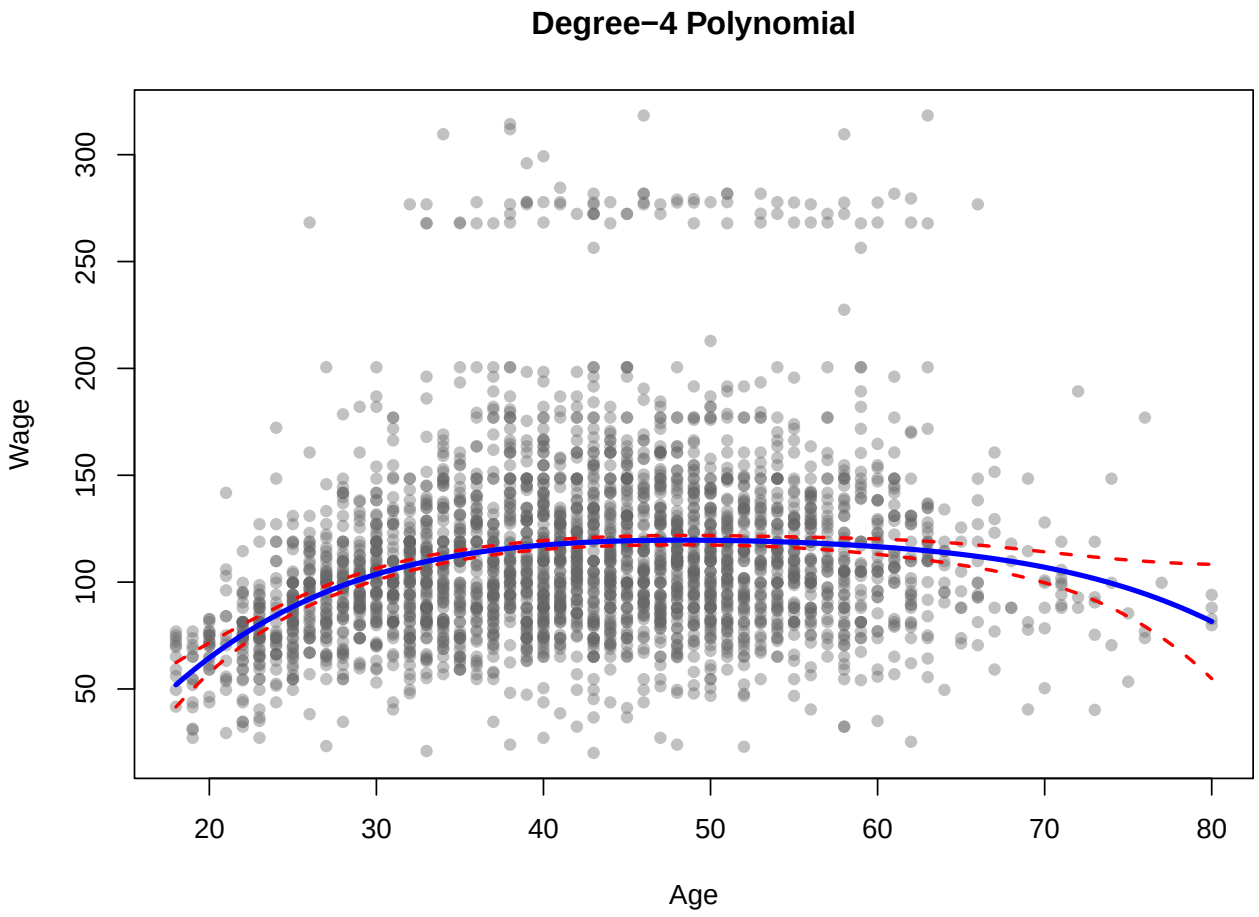
We define a helper function to plot the data, a fitted curve, and its 95% confidence interval for the **conditional mean**. We will reuse this function for regression splines and natural splines.

```
plot_wage_fit <- function(model, newdata, title) {
  if (inherits(model, "gam")) {
    # predict.gam does not support interval = "confidence".
    p <- predict(model, newdata = newdata, se.fit = TRUE)
    pred <- cbind(fit = p$fit,
                  lwr = p$fit - 1.96 * p$se.fit,
                  upr = p$fit + 1.96 * p$se.fit)
  } else {
    pred <- predict(model, newdata = newdata,
                    interval = "confidence", level = 0.95)
  }

  plot(Wage$age, Wage$wage,
       col = grDevices::adjustcolor("gray40", alpha.f = 0.4),
```

```
       pch = 16, xlab = "Age", ylab = "Wage", main = title)
  lines(newdata$age, pred[, "fit"], col = "blue", lwd = 3)
  lines(newdata$age, pred[, "lwr"], col = "red", lty = 2, lwd = 2)
  lines(newdata$age, pred[, "upr"], col = "red", lty = 2, lwd = 2)
  invisible(pred)
}
```

```
plot_wage_fit(poly_fit, age_df, "Degree-4 Polynomial")
```



The blue line shows the fitted mean wage; the red dashed lines indicate uncertainty about the estimated mean, **not** the interval containing 95% of individual wages. Transparent points help reveal areas with dense observations.

Splines

A regression spline models $f(x)$ with piecewise polynomials joined at **knots**. A common cubic spline can be written in the truncated-power basis as

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K} \gamma_k (x - \xi_k)_+^3,$$

where $\xi_1, \ldots, \xi_K$ are the interior knots and $(z)_+ = \max(z, 0)$.

Conceptually: `age` $\longrightarrow$ spline basis (depends on knots) $\longrightarrow$ linear regression $\longrightarrow$ predicted wage.

In R, `splines::bs()` constructs a B-spline basis. By default it makes cubic splines (`degree = 3`).

Regression splines with specified knots

```
bs_matrix <- bs(age, knots = c(25, 40, 60), degree = 3,
                intercept = TRUE)
dim(bs_matrix)
```

```
## [1] 3000    7
```

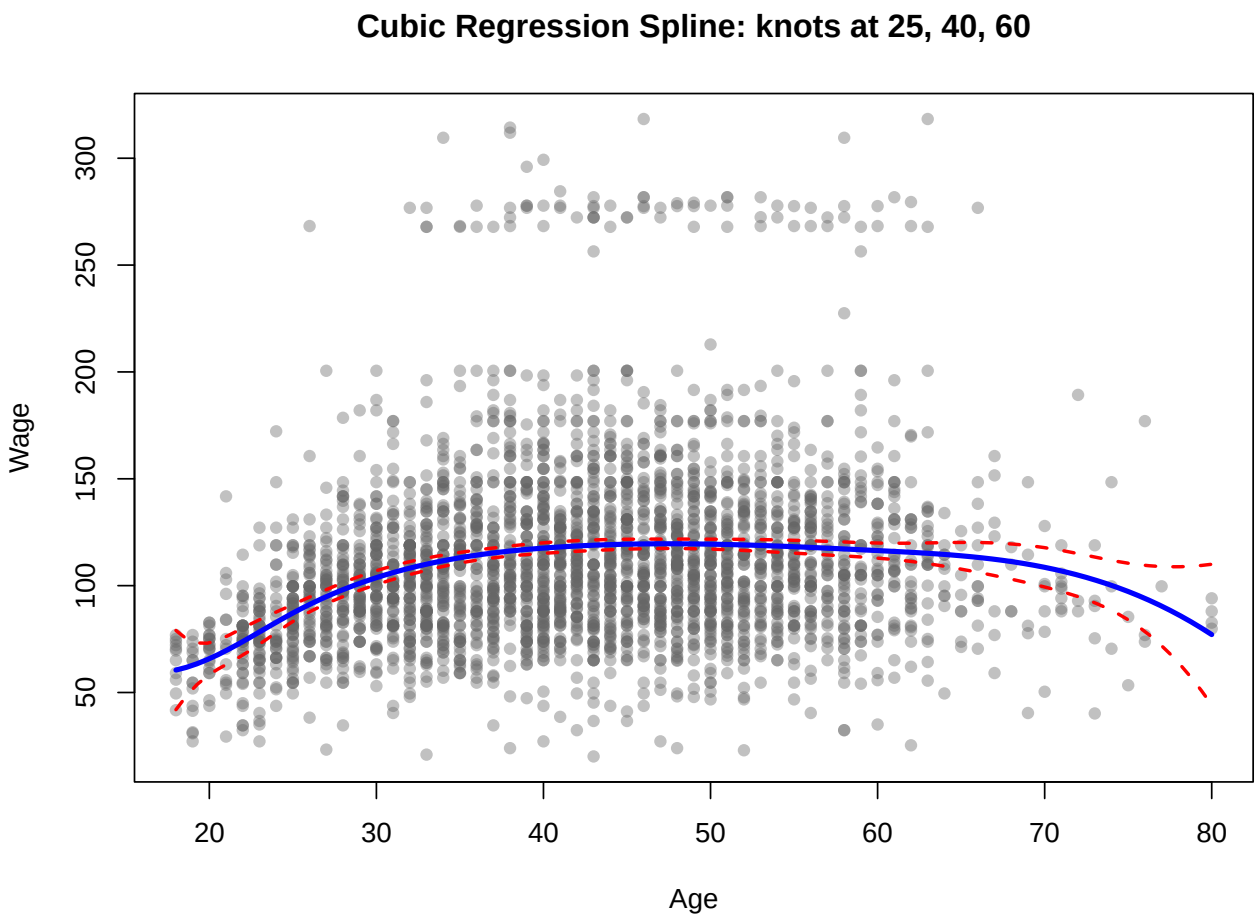
For a cubic spline with three interior knots, including all B-spline basis functions produces seven columns. In a regression model with a separate intercept, we ordinarily omit one basis column so that the design matrix is not redundant.

```
bs_fit <- lm(wage ~ bs(age, knots = c(25, 40, 60), degree = 3),
             data = Wage)
summary(bs_fit)
```

```
##
## Call:
## lm(formula = wage ~ bs(age, knots = c(25, 40, 60), degree = 3),
##     data = Wage)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -98.832 -24.537  -5.049  15.209 203.207
##
## Coefficients:
##                                             Estimate Std. Error t value
## (Intercept)                                   60.494      9.460   6.394
## bs(age, knots = c(25, 40, 60), degree = 3)1    3.980     12.538   0.317
## bs(age, knots = c(25, 40, 60), degree = 3)2   44.631      9.626   4.636
## bs(age, knots = c(25, 40, 60), degree = 3)3   62.839     10.755   5.843
## bs(age, knots = c(25, 40, 60), degree = 3)4   55.991     10.706   5.230
## bs(age, knots = c(25, 40, 60), degree = 3)5   50.688     14.402   3.520
## bs(age, knots = c(25, 40, 60), degree = 3)6   16.606     19.126   0.868
##                                             Pr(>|t|)
## (Intercept)                                 1.86e-10 ***
## bs(age, knots = c(25, 40, 60), degree = 3)1 0.750899
## bs(age, knots = c(25, 40, 60), degree = 3)2 3.70e-06 ***
## bs(age, knots = c(25, 40, 60), degree = 3)3 5.69e-09 ***
## bs(age, knots = c(25, 40, 60), degree = 3)4 1.81e-07 ***
## bs(age, knots = c(25, 40, 60), degree = 3)5 0.000439 ***
## bs(age, knots = c(25, 40, 60), degree = 3)6 0.385338
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 39.92 on 2993 degrees of freedom
## Multiple R-squared:  0.08642,    Adjusted R-squared:  0.08459
## F-statistic: 47.19 on 6 and 2993 DF,  p-value: < 2.2e-16
```

With three interior knots and the default `intercept = FALSE`, `bs()` contributes six spline columns; the regression adds the overall intercept.

```
plot_wage_fit(bs_fit, age_df, "Cubic Regression Spline: knots at 25, 40, 60")
```



Choosing spline complexity through degrees of freedom

We can specify the degrees of freedom (`df`) rather than interior knots. For a cubic B-spline with `intercept = FALSE`, `df = 6` results in three interior knots, placed at quantiles of the observed ages.

```
bs_df <- bs(age, df = 6)
attr(bs_df, "knots")
```

```
## [1] 33.75 42.00 51.00
```

```
bs_df_fit <- lm(wage ~ bs(age, df = 6), data = Wage)
summary(bs_df_fit)
```

```
##
## Call:
## lm(formula = wage ~ bs(age, df = 6), data = Wage)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -99.681 -24.403  -5.202  15.441 201.413
##
## Coefficients:
##                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)         56.314      7.258   7.759 1.17e-14 ***
## bs(age, df = 6)1    27.824     12.435   2.238   0.0253 *
## bs(age, df = 6)2    54.063      7.127   7.585 4.41e-14 ***
```

```
## bs(age, df = 6)3   65.828      8.323   7.909 3.62e-15 ***
## bs(age, df = 6)4   55.813      8.724   6.398 1.83e-10 ***
## bs(age, df = 6)5   72.131     13.745   5.248 1.65e-07 ***
## bs(age, df = 6)6   14.751     16.209   0.910   0.3629
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 39.91 on 2993 degrees of freedom
## Multiple R-squared:  0.08729,    Adjusted R-squared:  0.08546
## F-statistic: 47.71 on 6 and 2993 DF,  p-value: < 2.2e-16
```

The selected knots need not match the earlier manually chosen ages (25, 40, and 60). The quantile-based knots depend on the age distribution in the data.

Natural splines

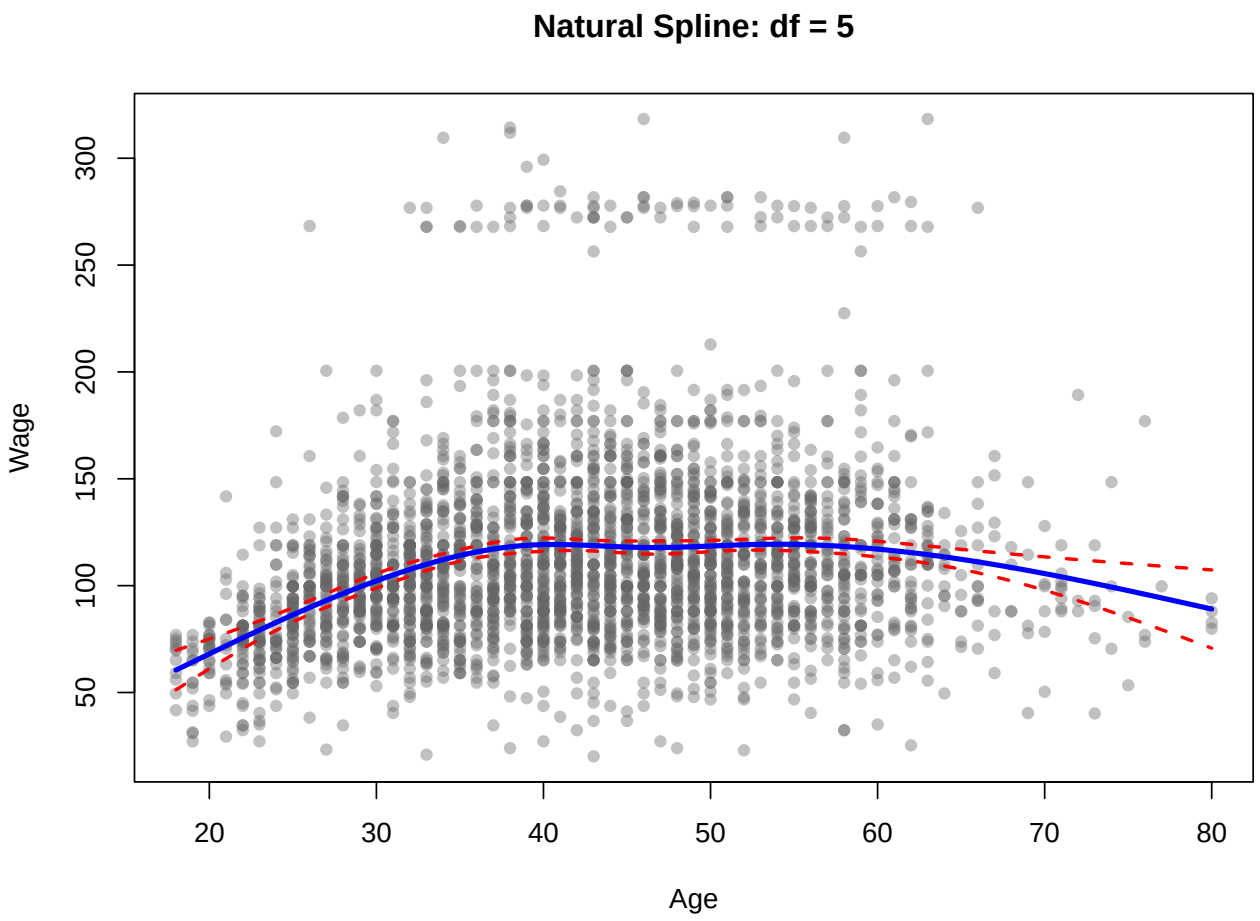
Natural cubic splines have additional boundary constraints: the fitted function is **linear beyond the boundary knots**. Equivalently, its second derivative is zero in those tails. This often produces more stable boundary behavior than an unrestricted cubic spline.

We use `splines::ns()` to fit a natural spline with five basis degrees of freedom (excluding the regression intercept).

```
ns_fit <- lm(wage ~ ns(age, df = 5), data = Wage)
summary(ns_fit)
```

```
##
## Call:
## lm(formula = wage ~ ns(age, df = 5), data = Wage)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -98.651 -24.303  -4.885  15.495 203.837
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)        60.475      4.708  12.844  < 2e-16 ***
## ns(age, df = 5)1   61.527      4.709  13.065  < 2e-16 ***
## ns(age, df = 5)2   55.691      5.717   9.741  < 2e-16 ***
## ns(age, df = 5)3   46.818      4.948   9.463  < 2e-16 ***
## ns(age, df = 5)4   83.204     11.918   6.982 3.58e-12 ***
## ns(age, df = 5)5    6.877      9.484   0.725    0.468
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 39.91 on 2994 degrees of freedom
## Multiple R-squared:  0.08683,    Adjusted R-squared:  0.08531
## F-statistic: 56.94 on 5 and 2994 DF,  p-value: < 2.2e-16
```

```
plot_wage_fit(ns_fit, age_df, "Natural Spline: df = 5")
```



Think about it: What would you expect to happen outside the observed age range under an unrestricted polynomial, a cubic regression spline, and a natural spline?

Generalized additive models (GAMs)

A GAM models the mean response with additive smooth functions:

$$E(Y \mid X_1, \ldots, X_p) = \beta_0 + f_1(X_1) + \cdots + f_p(X_p).$$

For a Gaussian response, `mgcv::gam()` estimates smooth terms such as `s(age)`. Rather than choosing the curvature by hand, penalized smoothing balances fit with a penalty on wiggliness, conceptually

$$\sum_{i=1}^{n} \{y_i - f(x_i)\}^2 + \lambda \int \{f''(x)\}^2 \, dx.$$

A **larger** λ places a stronger penalty on wiggliness, producing a smoother fitted curve; a **smaller** λ permits more flexibility. The exact penalty depends on the smoothing basis and model specification.

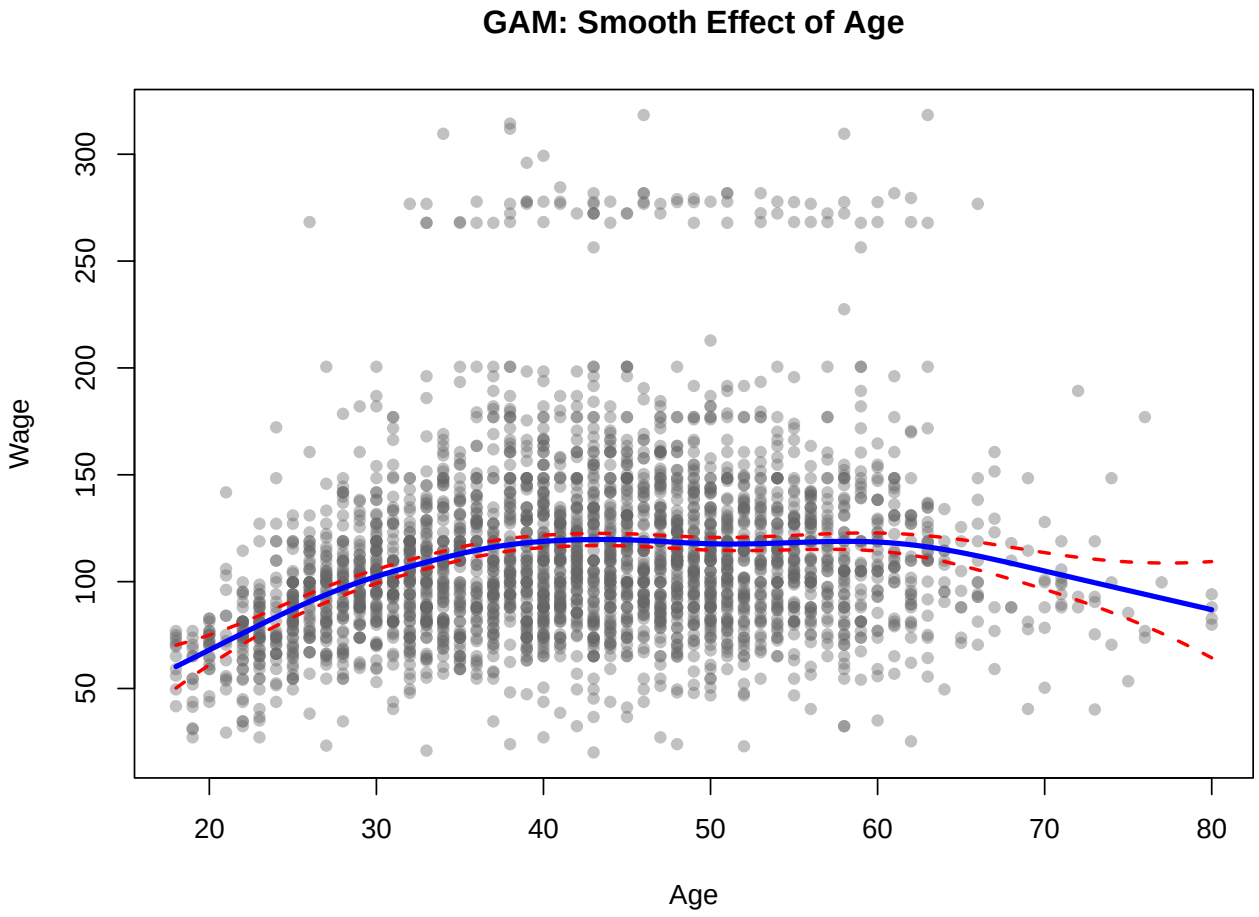
Fit a GAM using age

```
# k is the maximum basis dimension, not the chosen effective df.
gam_age <- gam(wage ~ s(age, k = 20), data = Wage,
```

```
               method = "REML")
summary(gam_age)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## wage ~ s(age, k = 20)
##
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 111.7036     0.7282   153.4   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##          edf Ref.df     F p-value
## s(age) 5.931  7.433 38.27  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.0864   Deviance explained = 8.83%
## -REML =  15319  Scale est. = 1590.7    n = 3000
```

```
plot_wage_fit(gam_age, age_df, "GAM: Smooth Effect of Age")
```



Unlike Python's `pygam`, R's `mgcv` works directly with a formula and data frame; we do not need to reshape `age` into a matrix.

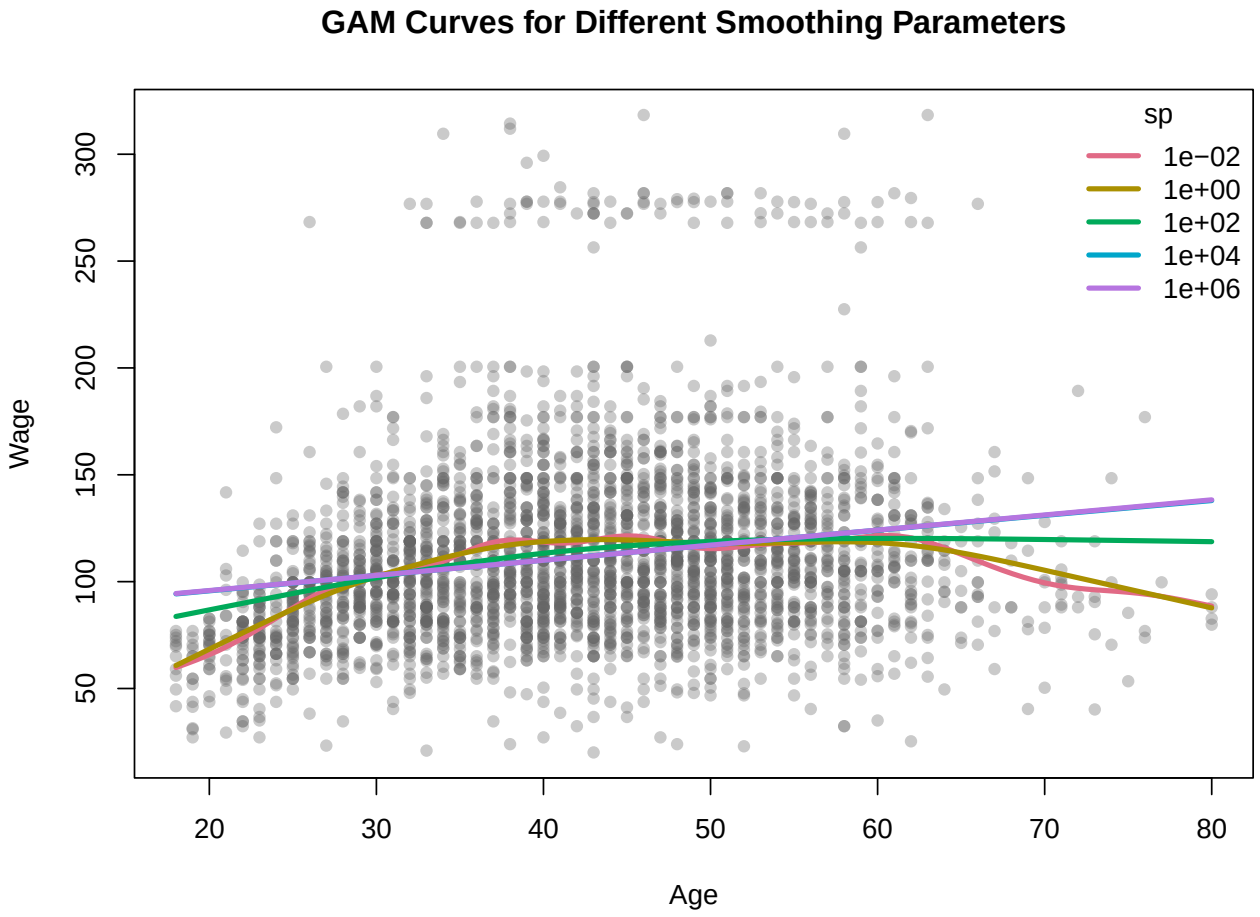
Exploring the smoothing parameter

The following fits use fixed smoothing parameters across several orders of magnitude. In `mgcv`, the `sp` argument supplies the smoothing parameter on the package's internal penalty scale; **its numerical value is not directly equivalent to `pygam`'s `lam`.**

```
sp_values <- 10^seq(-2, 6, length.out = 5)
cols <- grDevices::hcl.colors(length(sp_values), "Dark 3")

plot(age, y, pch = 16,
     col = grDevices::adjustcolor("gray40", alpha.f = 0.35),
     xlab = "Age", ylab = "Wage",
     main = "GAM Curves for Different Smoothing Parameters")

for (j in seq_along(sp_values)) {
  m <- gam(wage ~ s(age, k = 20), data = Wage,
           sp = sp_values[j], method = "REML")
  lines(age_grid, predict(m, newdata = age_df),
        col = cols[j], lwd = 3)
}
legend("topright", legend = format(sp_values, scientific = TRUE),
       col = cols, lwd = 3, title = "sp", bty = "n")
```



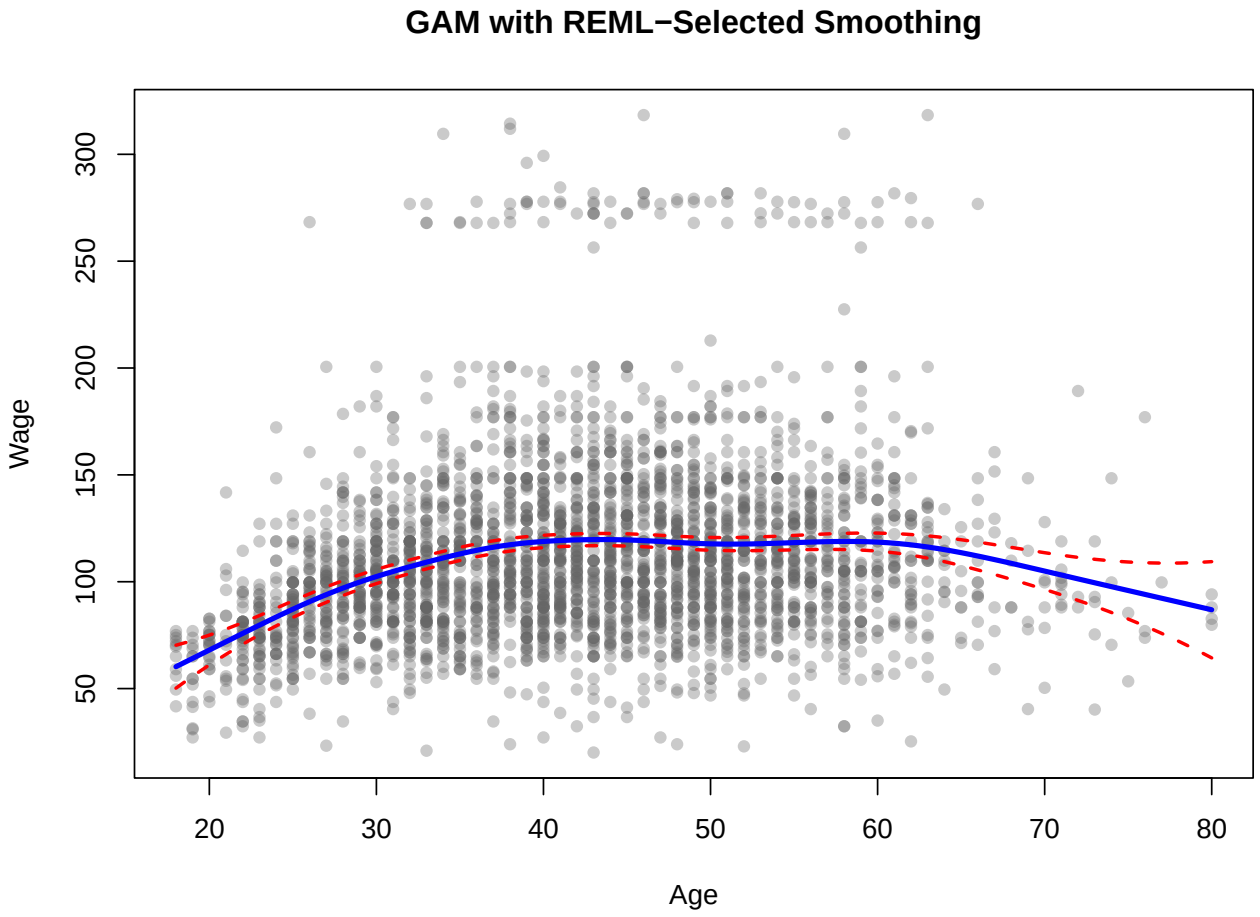
In practice, we commonly let `mgcv` **estimate** the amount of smoothing, for example using restricted maximum likelihood (REML). Unlike a manual grid search over the smoothing parameter, REML estimates it as part of fitting the model.

```
gam_opt <- gam(wage ~ s(age, k = 20), data = Wage,
               method = "REML")
summary(gam_opt)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## wage ~ s(age, k = 20)
##
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 111.7036     0.7282   153.4   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##          edf Ref.df     F p-value
## s(age) 5.931  7.433 38.27  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## R-sq.(adj) =  0.0864   Deviance explained = 8.83%
## -REML =  15319  Scale est. = 1590.7    n = 3000
```

```r
auto_pred <- predict(gam_opt, newdata = age_df, se.fit = TRUE)
plot(age, y, pch = 16,
     col = grDevices::adjustcolor("gray40", alpha.f = 0.35),
     xlab = "Age", ylab = "Wage",
     main = "GAM with REML-Selected Smoothing")
lines(age_grid, auto_pred$fit, col = "blue", lwd = 3)
lines(age_grid, auto_pred$fit + 1.96 * auto_pred$se.fit,
      col = "red", lty = 2, lwd = 2)
lines(age_grid, auto_pred$fit - 1.96 * auto_pred$se.fit,
      col = "red", lty = 2, lwd = 2)
```



The **effective degrees of freedom (edf)** reported in `summary()` describe the complexity of the fitted smooth. An edf near one suggests an approximately linear effect; larger values indicate increasing nonlinear flexibility.

Additive models with several terms

We now use three predictors:

$$E(\text{wage} \mid \text{age, year, education}) = \beta_0 + f_1(\text{age}) + f_2(\text{year}) + \text{effect of education.}$$

The model is additive: each predictor has its own contribution. Strong interactions must be added explicitly if they are important.

Approach 1: Natural spline bases and ordinary linear regression

This is an ordinary linear model in an expanded set of basis functions. R creates the spline columns and education dummy variables internally.

```
Wage$education <- as.factor(Wage$education)

additive_lm <- lm(wage ~ ns(age, df = 4) + ns(year, df = 5) + education,
                  data = Wage)
summary(additive_lm)
```

```
##
## Call:
## lm(formula = wage ~ ns(age, df = 4) + ns(year, df = 5) + education,
##     data = Wage)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -119.58  -19.91   -3.24   14.44  212.25
##
## Coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                    43.801      4.383   9.993  < 2e-16 ***
## ns(age, df = 4)1               46.446      3.732  12.446  < 2e-16 ***
## ns(age, df = 4)2               28.935      3.884   7.449 1.22e-13 ***
## ns(age, df = 4)3               63.672      9.231   6.898 6.43e-12 ***
## ns(age, df = 4)4               10.967      7.650   1.434   0.1518
## ns(year, df = 5)1               1.837      3.177   0.578   0.5631
## ns(year, df = 5)2              10.441      3.790   2.755   0.0059 **
## ns(year, df = 5)3               2.002      3.399   0.589   0.5559
## ns(year, df = 5)4               9.605      4.053   2.370   0.0178 *
## ns(year, df = 5)5               5.899      2.419   2.438   0.0148 *
## education2. HS Grad            10.932      2.432   4.496 7.20e-06 ***
## education3. Some College       23.397      2.562   9.131  < 2e-16 ***
## education4. College Grad       38.165      2.547  14.986  < 2e-16 ***
## education5. Advanced Degree    62.570      2.763  22.646  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 35.18 on 2986 degrees of freedom
## Multiple R-squared:  0.2923, Adjusted R-squared:  0.2892
## F-statistic: 94.86 on 13 and 2986 DF,  p-value: < 2.2e-16
```

```
# See the combined model matrix, including spline bases and dummies:
X_bh <- model.matrix(additive_lm)
dim(X_bh)
```

```
## [1] 3000   14
```

```
head(X_bh[, seq_len(min(8, ncol(X_bh)))])
```

```
##        (Intercept) ns(age, df = 4)1 ns(age, df = 4)2 ns(age, df = 4)3
## 231655           1       0.00000000       0.00000000       0.00000000
```

```
## 86582            1       0.01731602      -0.13795411       0.31872157
## 161300           1       0.75108560       0.16605047       0.09131609
## 155159           1       0.78017192       0.07222489       0.11002552
## 11443            1       0.52933205       0.38879374       0.13708340
## 376662           1       0.34484721       0.49710028       0.19449432
##        ns(age, df = 4)4 ns(year, df = 5)1 ns(year, df = 5)2 ns(year, df = 5)3
## 231655       0.00000000         0.3333333        0.58333333        0.08012821
## 86582       -0.18076746         0.1666667        0.00000000       -0.12403473
## 161300      -0.05061290         0.0000000        0.00000000        0.00000000
## 155159      -0.06235889         0.0000000        0.00000000        0.00000000
## 11443       -0.05540437         0.7083333        0.12500000       -0.03100868
## 376662      -0.03644180         0.0000000        0.08333333        0.52243590
```

`model.matrix()` reveals the expanded columns. R automatically handles the intercept and factor coding, avoiding the need to concatenate multiple design matrices manually.

Partial dependence-style plots

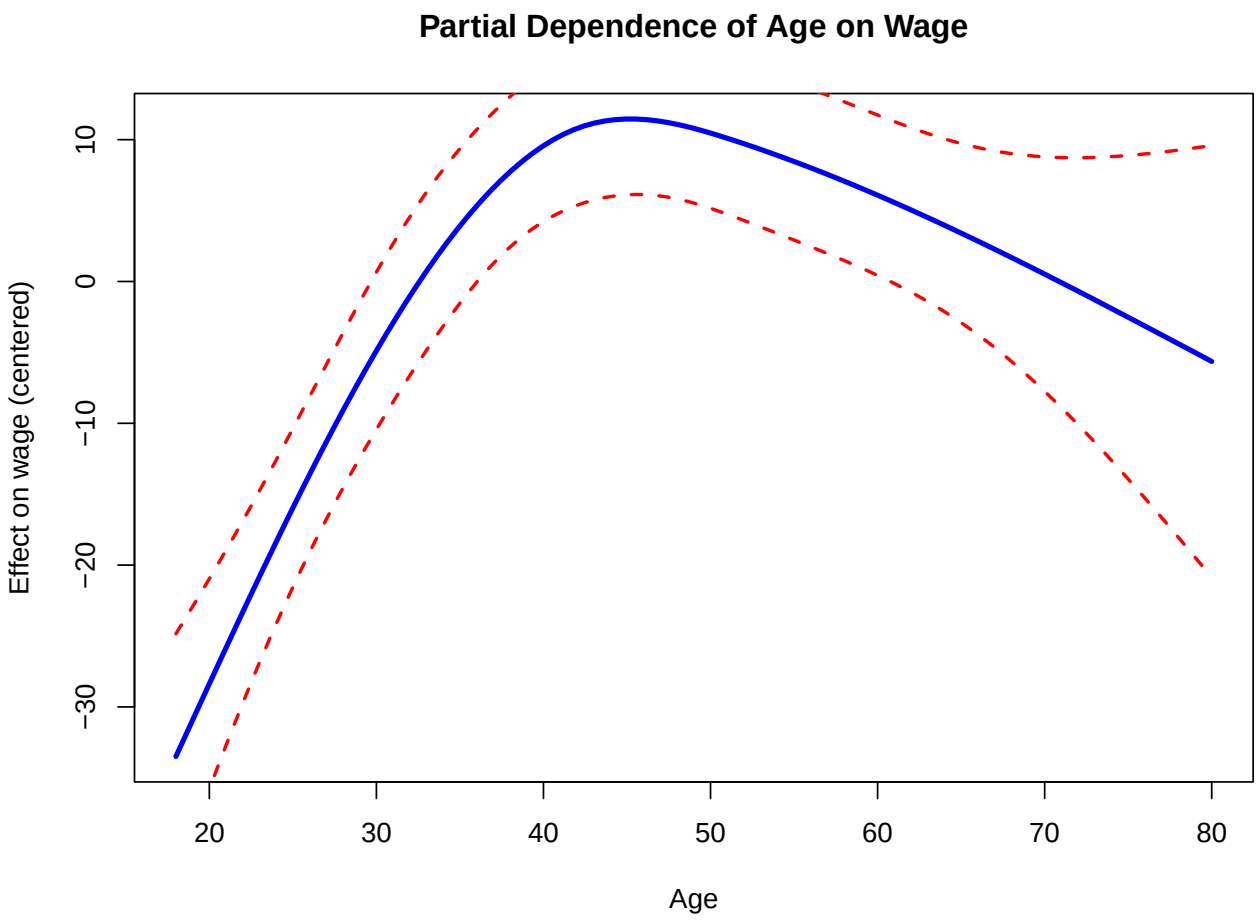
For each term, we vary one predictor over a grid while holding other predictors at reference values. We center the resulting curve so we can visualize the relative change in fitted wage. These curves represent **adjusted fitted effects**, not necessarily causal effects.

```
reference_year <- median(Wage$year)
reference_education <- levels(Wage$education)[1]

age_new <- data.frame(
  age = age_grid,
  year = reference_year,
  education = factor(reference_education, levels = levels(Wage$education))
)

p_age <- predict(additive_lm, newdata = age_new,
                 interval = "confidence")
center_age <- mean(p_age[, "fit"])
p_age_centered <- sweep(p_age, 2, center_age, FUN = "-")

plot(age_grid, p_age_centered[, "fit"], type = "l", col = "blue", lwd = 3,
     xlab = "Age", ylab = "Effect on wage (centered)",
     main = "Partial Dependence of Age on Wage")
lines(age_grid, p_age_centered[, "lwr"], col = "red", lty = 2, lwd = 2)
lines(age_grid, p_age_centered[, "upr"], col = "red", lty = 2, lwd = 2)
```



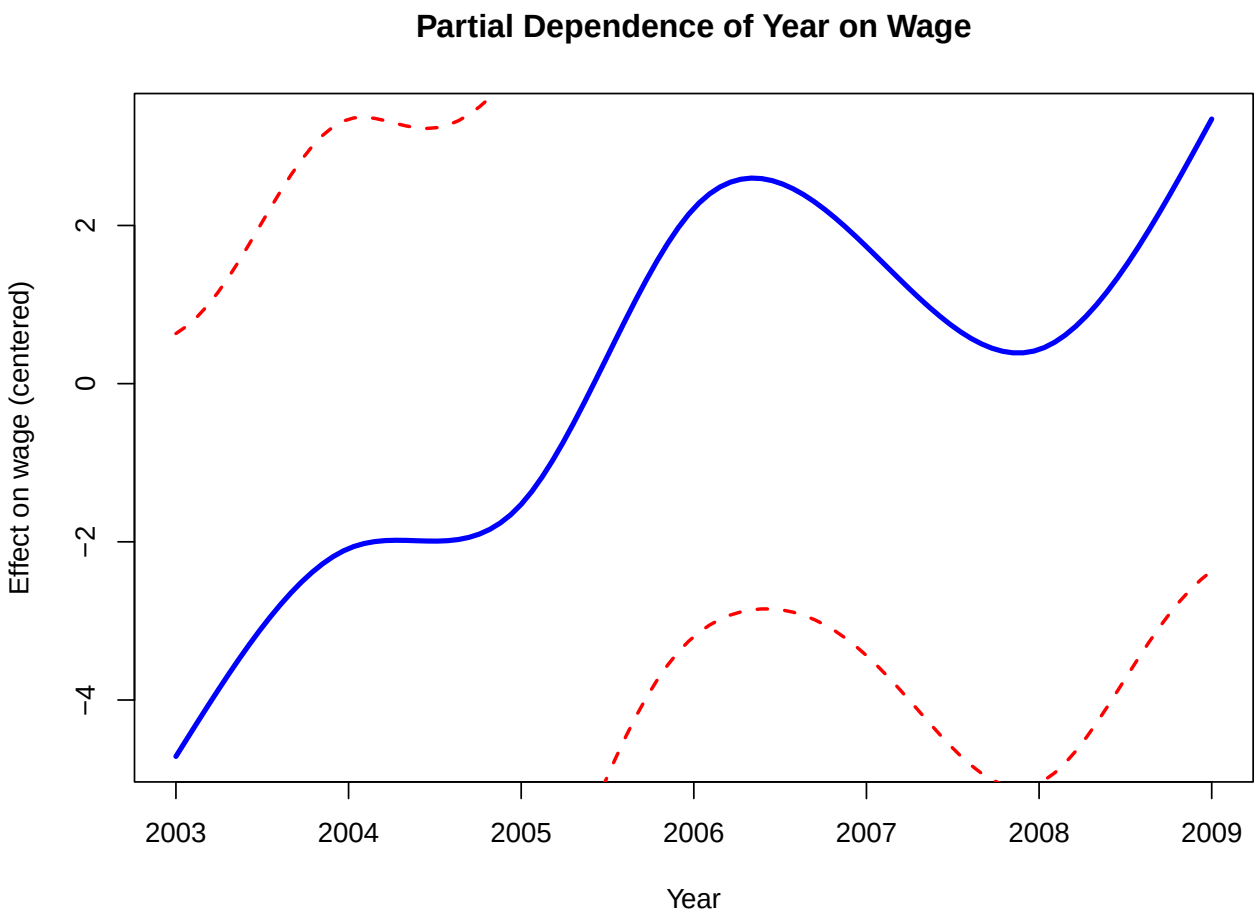
We can follow the same process for `year` while holding `age` and `education` fixed.

```
year_grid <- seq(min(Wage$year), max(Wage$year), length.out = 100)

year_new <- data.frame(
  age = median(Wage$age),
  year = year_grid,
  education = factor(reference_education, levels = levels(Wage$education))
)

p_year <- predict(additive_lm, newdata = year_new,
                  interval = "confidence")
center_year <- mean(p_year[, "fit"])
p_year_centered <- sweep(p_year, 2, center_year, FUN = "-")

plot(year_grid, p_year_centered[, "fit"], type = "l", col = "blue", lwd = 3,
     xlab = "Year", ylab = "Effect on wage (centered)",
     main = "Partial Dependence of Year on Wage")
lines(year_grid, p_year_centered[, "lwr"], col = "red", lty = 2, lwd = 2)
lines(year_grid, p_year_centered[, "upr"], col = "red", lty = 2, lwd = 2)
```



Because the model is additive, the change in the fitted value as one predictor varies is the same regardless of the fixed reference values of the other predictors; centering removes that constant level.

Approach 2: Fit the additive model using `mgcv::gam()`

Instead of manually specifying unpenalized natural-spline bases, we can fit penalized smooth terms and let REML select their smoothness.

```
gam_full <- gam(wage ~ s(age, k = 20) + s(year, k = 7) + education,
                data = Wage, method = "REML")
summary(gam_full)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## wage ~ s(age, k = 20) + s(year, k = 7) + education
##
## Parametric coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                    85.445      2.153  39.694  < 2e-16 ***
## education2. HS Grad            10.983      2.428   4.523 6.35e-06 ***
## education3. Some College       23.530      2.558   9.199  < 2e-16 ***
## education4. College Grad       38.191      2.543  15.020  < 2e-16 ***
## education5. Advanced Degree    62.573      2.759  22.681  < 2e-16 ***
```

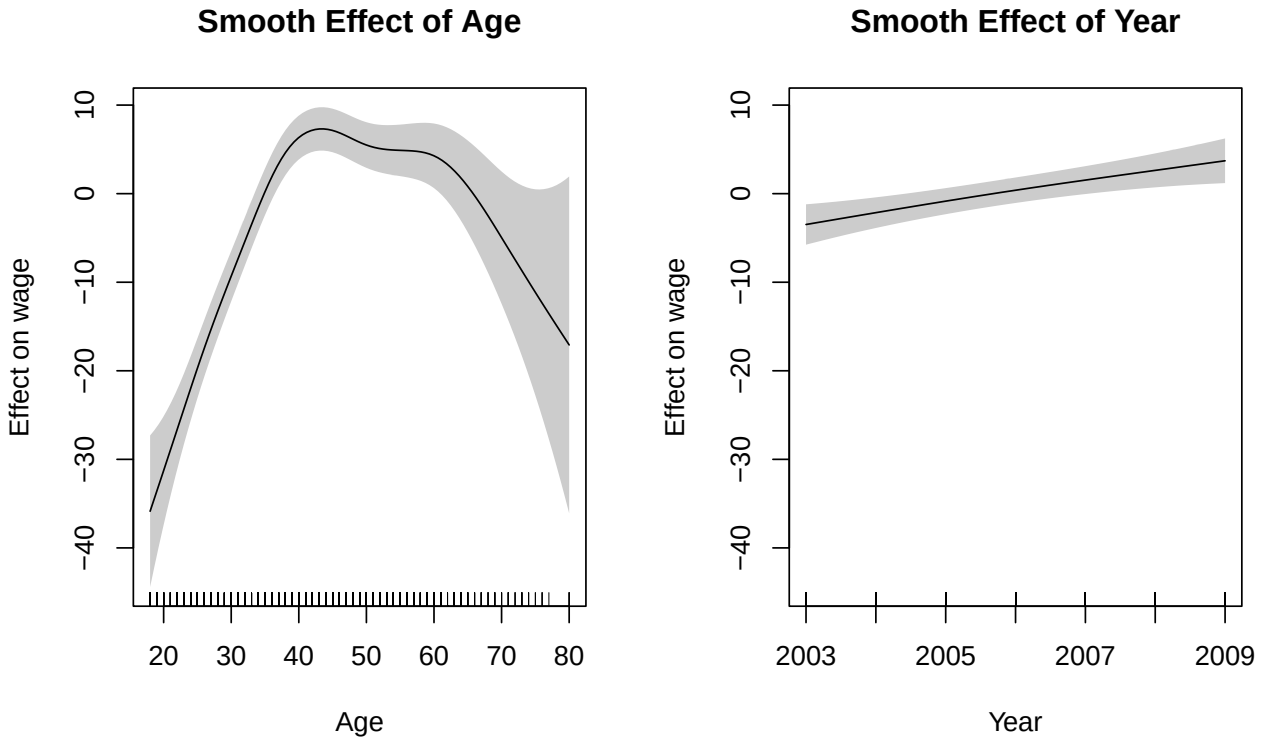
```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##           edf Ref.df     F p-value
## s(age)  5.344  6.717 33.73 < 2e-16 ***
## s(year) 1.166  1.313 10.10 0.00047 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =   0.29   Deviance explained = 29.3%
## -REML =  14930  Scale est. = 1235.6    n = 3000
```

The terms `s(age)` and `s(year)` add smooth functions. Because `year` has only a small number of distinct observed values, we use a smaller basis dimension for it. `education` enters as a categorical (parametric) effect; it does not receive a smoothness penalty.

Partial effects of age and year

Unlike `predict(..., type = "response")`, `predict(..., type = "terms")` isolates the additive contributions on the model's linear predictor scale. We can also use `plot()` to show the smooth terms directly.

```
par(mfrow = c(1, 2))
plot(gam_full, select = 1, shade = TRUE, seWithMean = TRUE,
     xlab = "Age", ylab = "Effect on wage",
     main = "Smooth Effect of Age")
plot(gam_full, select = 2, shade = TRUE, seWithMean = TRUE,
     xlab = "Year", ylab = "Effect on wage",
     main = "Smooth Effect of Year")
```




```
par(mfrow = c(1, 1))
```

If you want a less flexible model, you can reduce the basis dimension (`k`) and compare the effective degrees of freedom. Keep in mind: `k` sets an upper limit on complexity and is **not** the same as directly setting the edf to four.

```
gam_simpler <- gam(wage ~ s(age, k = 5) + s(year, k = 5) + education,
                   data = Wage, method = "REML")
summary(gam_simpler)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## wage ~ s(age, k = 5) + s(year, k = 5) + education
##
## Parametric coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                    85.561      2.152  39.758  < 2e-16 ***
## education2. HS Grad            10.887      2.429   4.483 7.64e-06 ***
## education3. Some College       23.380      2.557   9.144  < 2e-16 ***
## education4. College Grad       38.010      2.541  14.959  < 2e-16 ***
## education5. Advanced Degree    62.494      2.759  22.648  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##           edf Ref.df     F  p-value
## s(age)  3.612  3.911 56.82  < 2e-16 ***
## s(year) 1.138  1.262 10.10 0.000561 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.289   Deviance explained = 29.1%
## -REML =  14931  Scale est. = 1237.3    n = 3000
```

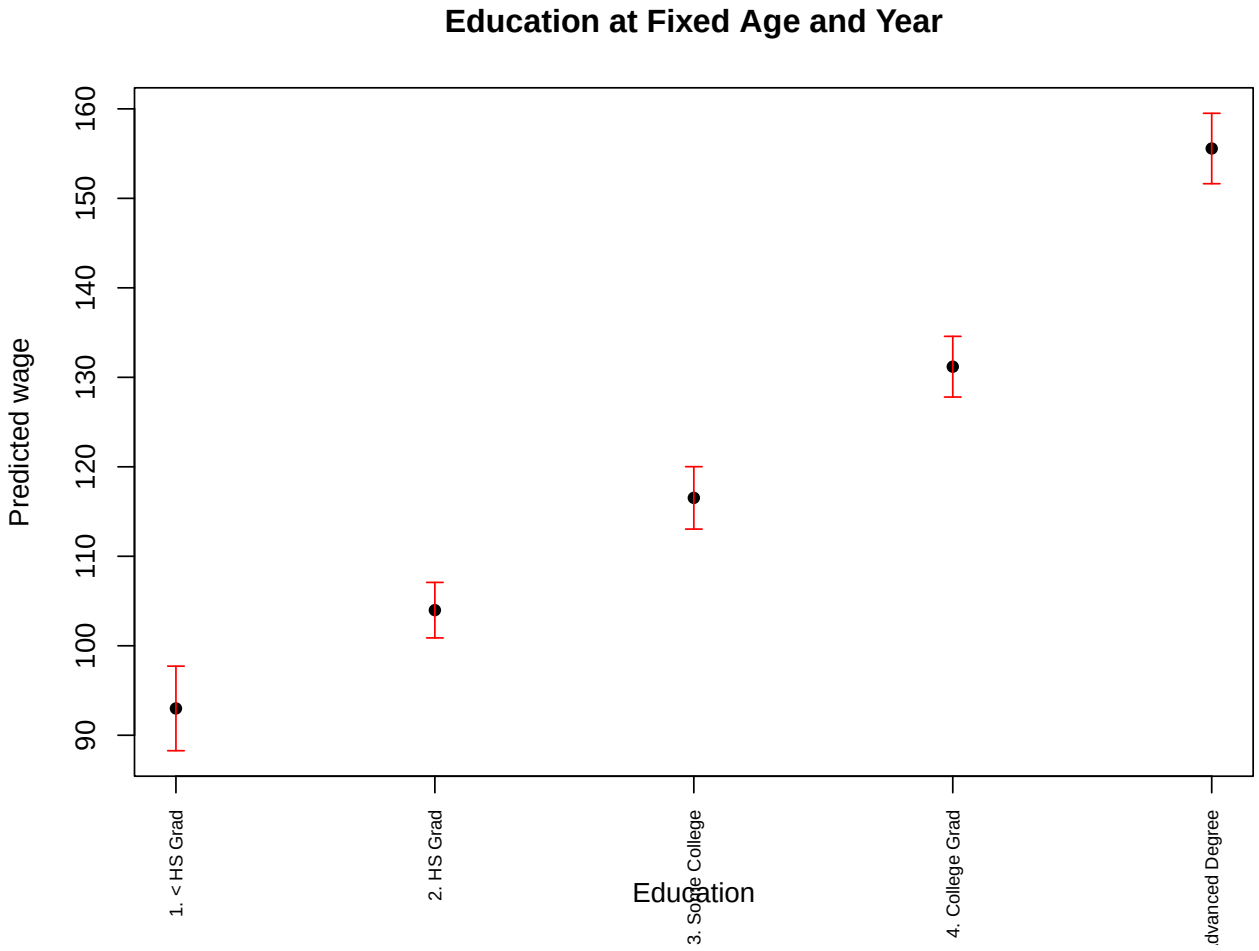
Education effects

Since `education` is a factor, we examine fitted wages at a common reference age and year for its levels.

```
educ_new <- data.frame(
  age = median(Wage$age),
  year = median(Wage$year),
  education = factor(levels(Wage$education), levels = levels(Wage$education))
)

educ_fitted <- predict(gam_full, newdata = educ_new, se.fit = TRUE)
plot(seq_along(educ_fitted$fit), educ_fitted$fit,
     xaxt = "n", pch = 16, xlab = "Education",
     ylab = "Predicted wage", main = "Education at Fixed Age and Year",
     ylim = range(c(educ_fitted$fit - 1.96 * educ_fitted$se.fit,
                    educ_fitted$fit + 1.96 * educ_fitted$se.fit)))
axis(1, at = seq_along(educ_fitted$fit),
     labels = levels(Wage$education), las = 2, cex.axis = 0.7)
arrows(seq_along(educ_fitted$fit),
       educ_fitted$fit - 1.96 * educ_fitted$se.fit,
```

```
        seq_along(educ_fitted$fit),
        educ_fitted$fit + 1.96 * educ_fitted$se.fit,
        angle = 90, code = 3, length = 0.05, col = "red")
```



The fitted values above include the intercept and the fixed reference contributions of age and year, making the education-level comparison easier to interpret.

LOWESS (local regression)

LOWESS fits a local regression around each target value x_0, giving observations closer to x_0 more weight. A local linear fit can be written as

$$\min_{a,b} \sum_{i=1}^{n} w_i(x_0)\{y_i - a - b(x_i - x_0)\}^2,$$

with fitted value $\hat{f}(x_0) = \hat{a}$. The weights decrease with distance from x_0.

The **span** controls the fraction of data used in a local fit:

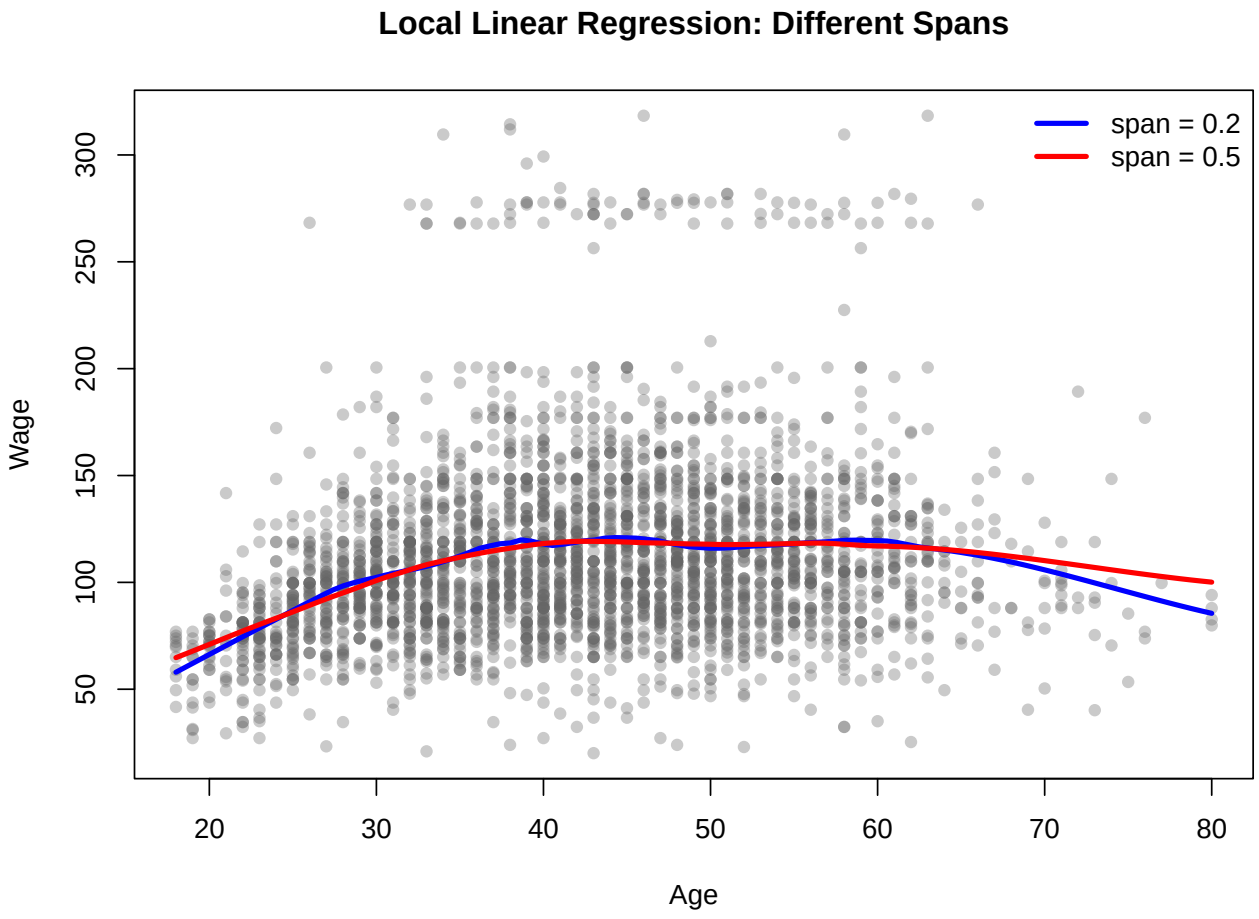
- `span = 0.2`: narrower neighborhoods, usually more flexible.
- `span = 0.5`: wider neighborhoods, usually smoother.

Base R's `loess()` implements a closely related local polynomial method. We set `degree = 1` to use local linear fits, and `family = "gaussian"` to avoid an additional robust reweighting step.

```r
loess_02 <- loess(wage ~ age, data = Wage,
                  span = 0.2, degree = 1, family = "gaussian")
loess_05 <- loess(wage ~ age, data = Wage,
                  span = 0.5, degree = 1, family = "gaussian")

pred_02 <- predict(loess_02, newdata = age_df)
pred_05 <- predict(loess_05, newdata = age_df)

plot(age, y, pch = 16,
     col = grDevices::adjustcolor("gray40", alpha.f = 0.35),
     xlab = "Age", ylab = "Wage",
     main = "Local Linear Regression: Different Spans")
lines(age_grid, pred_02, col = "blue", lwd = 3)
lines(age_grid, pred_05, col = "red", lwd = 3)
legend("topright", legend = c("span = 0.2", "span = 0.5"),
       col = c("blue", "red"), lwd = 3, bty = "n")
```



Note: `stats::lowess()` is also available in R, but `loess()` offers a formula interface and explicit local polynomial degree, making it more convenient here.

Summary and a question to think about

- **Polynomial regression:** a global polynomial basis. Easy to fit, but high degrees can overfit or behave strangely near boundaries.

- **Regression splines:** piecewise polynomials joined at knots; local flexibility is controlled by knot placement and basis size.
- **Natural splines:** cubic splines constrained to have linear tails, often yielding more stable boundary behavior.
- **GAMs / smoothing splines:** additive functions with penalized smoothness; smoothing parameters control wiggliness, and effective degrees of freedom describe fitted complexity.
- **LOWESS:** local regression fits around target values, with the span determining neighborhood size.

How should we choose among models?

There's no single best answer—here are a few practical hints:

1. What's the goal? - Interpretation (clear partial effects) → splines / GAMs - Pure prediction → choose complexity by cross-validation (degree/df/lam)

2. Do you care about boundary behavior / extrapolation? -Natural splines or stronger regularization often behave better near the edges.

3. How many predictors do you have? - Many predictors → GAMs are a natural next step (additive structure) - LOWESS is mostly used for 1D smoothing.

4.How much data do you have? - More data supports more flexibility; with small data, regularization matters more.

Optional exercise: Validation split

Try two natural-spline models of different complexity. Compare their training and validation mean squared errors. What happens to the shape of the fitted curve as `df` increases?

```
set.seed(442)
train_ids <- sample(seq_len(nrow(Wage)), size = floor(0.7 * nrow(Wage)))
train <- Wage[train_ids, ]
test  <- Wage[-train_ids, ]

# Fix the boundary knots to the observed age range, so both training
# and validation ages are within the spline's declared support.
all_boundaries <- range(Wage$age)

fit_4 <- lm(wage ~ ns(age, df = 4, Boundary.knots = all_boundaries), data = train)
fit_8 <- lm(wage ~ ns(age, df = 8, Boundary.knots = all_boundaries), data = train)

mse <- function(actual, predicted) mean((actual - predicted)^2)

results <- data.frame(
  df = c(4, 8),
  training_MSE = c(mse(train$wage, fitted(fit_4)),
                   mse(train$wage, fitted(fit_8))),
  validation_MSE = c(mse(test$wage, predict(fit_4, newdata = test)),
                     mse(test$wage, predict(fit_8, newdata = test)))
)
results
```

```
##   df training_MSE validation_MSE
## 1  4     1590.315       1593.548
## 2  8     1584.506       1594.731
```

Discussion: Does the model with the smaller training MSE also have the smaller validation MSE? Why might the answer differ across random splits?